

FairKalah Without Empty Captures

Phong T. Pham and Todd W. Neller

Gettysburg College, Gettysburg PA 17325, USA

thuanphongpham@gmail.com, tneller@gettysburg.edu

<http://cs.gettysburg.edu/~tneller/>

Abstract. Kalah (a.k.a. Mancala) is a two-player game of perfect information that has been a popular game for over half a century despite a strong first player advantage. Previous AI research concerns Kalah with empty captures, yet there is evidence that original rules disallow empty captures. In this paper, we present the first analysis of Kalah without empty captures, initial game states that are fair, as well as optimal play insights from analysis of optimal and suboptimal states.

1 Introduction

Kalah (a.k.a. Kalaha), usually called “Mancala” in the United States, is a Mancala game variant invented in 1940 by William Julius Champion and patented in 1955 as US Patent 2,720,362 [4]. In the Mancala games family, it is most closely related to Dakon [12, p. 75] from Java. Despite being a perfect information game with a strong first player advantage [6], Kalah has enjoyed popularity in the United States and Germany through its history. In this paper, we share initial game states that are fair¹ and provide insights to game play from analysis of optimal and suboptimal states.

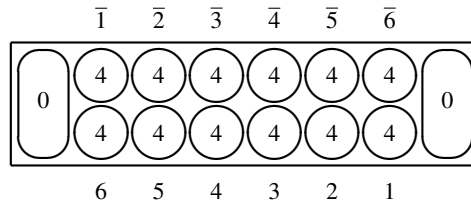


Fig. 1: Kalah initial board state with 4 pieces per play pit

Kalah (a.k.a. Mancala) is played on a rectangular board as depicted in Figure 1 with 6 play pits for each player along the long side and 1 score pit (“Kalah”) for each player on the player’s right-hand end of the board. In the standard initial board state, each of the play pits starts with 4 pieces (a.k.a. seeds, counters) per pit. In contrast to prior

¹ In this paper, we use “fair” in the sense that two perfect players would draw the game. There is no game-theoretic advantage for either player.

rules, we label play pits by the number of pits they are clockwise from that player’s score pit. Pits of the second player (a.k.a. “north player”) are notated with overbars. Numbers depicted within pits indicate the number of pieces in that pit².

On a player’s turn, the player selects one of their play pits containing pieces, removes all pieces from that play pit, and redistributes (“sows”) them counterclockwise, one piece per play/score pit, and skipping their opponent’s score pit. If the last piece redistributed lands in their score pit, the player takes another turn, i.e. gets a free move; otherwise, it becomes the opponent’s turn. If the last piece redistributed lands in an empty play pit of the player across from a non-empty opponent play pit, then that piece captures: both that capturing piece and pieces in the opponent’s opposite play pit are removed from the pits and placed into the player’s score pit. There is some controversy over whether “empty captures” (a.k.a. “null captures”) were intended, as they were disallowed in Dakon. A strict reading of the patent makes no requirement for the number of opponent pieces opposite, and thus prior work [6,9] has allowed empty captures. However, there are many who play Kalah *disallowing empty captures* and there is evidence that empty captures were not Champion’s intent for Kalah³ In this paper, we will examine Kalah *without empty captures*.

The game ends when, at the end of a turn, there are no pieces left in one player’s play pits. Their opponent scores any remaining pieces in play pits. The player with more pieces in their score pit wins. If the number of pieces in the score pits are the same, the players draw (i.e. tie) the game.

While boards and pieces are readily available commercially, this game has often been played with egg cartons and bowls as play and score pits, respectively. Glass beads, beans, pieces, and shells are just some of the materials used throughout the world to play Mancala games.

Featuring simple rules and accessibility of materials, Kalah has been successfully popular, but with a little play, one can easily discern that opening with plays 4 and 1 put the second player at a significant advantage. Irving, Donkers, and Uiterwijk computed that optimal play with empty captures for the standard initial board results in a first player win by 10 points (29-19). Without empty captures, we compute an optimal first player win by 8 points (28-20; see Table 1). In our notation, one line of optimal play of Figure 1 is as follows: 41, $\overline{56}$, 5, $\overline{3}$, 12, $\overline{1}$, 6245, $\overline{2}$, 3, $\overline{3}$, 5, $\overline{21}$, 2, $\overline{5}$, 4, $\overline{6}$, 5, $\overline{214}$, 3, $\overline{5}$, 21, $\overline{6}$, 6, $\overline{4}$. Using this notation has the convenience that when there are free moves on a turn (e.g. 6245), the number of pieces moved is the same as the pit label for all except the final move.

In this paper, we present initial states that are *fair*, i.e. that result in a draw between two optimal players. *FairKalah* is the name we use for Kalah played from a fair initial state. Additionally, we analyze optimal play data in order to yield optimal play insights.

² Thus, a potential free move is easily seen where the number of pieces matches our numeric label for that pit.

³ For example, the first Kalah AI by Roland Silver [13] disallowed empty captures and was played by William Champion [1].

1.1 Example Games under Perfect Play

To provide intuition for how the difference in the empty-capture rule influences game-play. The notation $m(n)$ represents a configuration with m pits and n pieces per pit, while the “Game Value” indicates the score differential for the first player (W = Win, L = Loss, D = Draw).

$m(n)$	Game Value	Perfect Play
4(1)	2 (W)	12, $\bar{3}$, 4, $\bar{121}$
4(2)	6 (W)	1, $\bar{1}$, 214, $\bar{213}$, 3121
4(3)	6 (W)	32, $\bar{3}$, 41, $\bar{4}$, 4, $\bar{13}$, 3, $\bar{12}$, 4, $\bar{1}$
4(4)	2 (W)	43, $\bar{3}$, 3, $\bar{4}$, 4, $\bar{2}$, 2, $\bar{2}$, 34, $\bar{4}$, 1, $\bar{3}$, 4, $\bar{14}$, 1213, $\bar{13}$, 12
4(5)	4 (W)	4, $\bar{4}$, 4, $\bar{2}$, 4, $\bar{3}$, 4, $\bar{2}$, 31, $\bar{21}$, 1413, $\bar{3}$, 12, $\bar{2}$, 3, $\bar{4321}$, 3, $\bar{4}$
4(6)	0 (D)	3, $\bar{4}$, 2, $\bar{1}$, 3114, $\bar{43}$, 2, $\bar{4}$, 1, $\bar{4}$, 4, $\bar{3}$, 2, $\bar{2}$, 21, $\bar{311412}$, 23, $\bar{1}$, 24, $\bar{24}$, 2, $\bar{13}$, 4, $\bar{12}$, 3, $\bar{1}$
5(1)	0 (D)	5, $\bar{5}$, 4, $\bar{4}$, 1213, $\bar{23}$, 12, $\bar{2}$, 1
5(2)	2 (W)	24, $\bar{1}$, 1, $\bar{4}$, 35, $\bar{235}$, 4, $\bar{4}$, 1213, $\bar{23}$, 12, $\bar{2}$, 1
5(3)	6 (W)	31, $\bar{2}$, 2, $\bar{53}$, 14521, $\bar{5}$, 3, $\bar{24}$, 24, $\bar{2}$, 2, $\bar{1}$, 35, $\bar{4}$, 24, $\bar{5}$, 3, $\bar{3}$, 2, $\bar{2}$, 1
5(4)	10 (W)	43, $\bar{53}$, 51, $\bar{5}$, 4, $\bar{3}$, 3, $\bar{4}$, 12, $\bar{1}$, 15, $\bar{5}$, 32, $\bar{4}$, 1, $\bar{5}$, 4, $\bar{43}$, 12, $\bar{2}$, 1, $\bar{5}$, 3, $\bar{4}$, 5, $\bar{32}$, 21
6(1)	2 (W)	12, $\bar{5}$, 6, $\bar{4}$, 5, $\bar{1213}$, 4, $\bar{12}$, 3121
6(2)	6 (W)	24, $\bar{24}$, 6, $\bar{6}$, 5, $\bar{5}$, 4, $\bar{1}$
6(3)	2 (W)	2, $\bar{32}$, 1, $\bar{46}$, 6, $\bar{4}$, 412, $\bar{231}$, 3, $\bar{6}$, 4, $\bar{2}$, 5, $\bar{5}$, 1, $\bar{5}$, 23, $\bar{6}$, 12, $\bar{5}$, 1
6(4)	8 (W)	41, $\bar{56}$, 5, $\bar{3}$, 12, $\bar{1}$, 6245, $\bar{2}$, 3, $\bar{3}$, 5, $\bar{21}$, 2, $\bar{5}$, 4, $\bar{6}$, 5, $\bar{214}$, 3, $\bar{5}$, 21, $\bar{6}$, 6, $\bar{4}$

Table 1: Games under perfect play for various pit and piece configurations.

2 Algorithmic Approach

Our approach to computing fair initial states for FairKalah is based on Java code we ported from Irving’s original C code used for the results of [6]. Game logic was modified to disallow empty captures. We computed a 24-piece endgame database (1.22 GB), so search could terminate when half of the pieces had been scored. The standard initial board has 48 pieces with 4 pieces per play pit. At the top level of our algorithm, we enumerated all possible ways that one could move one or two of these pieces to different pits, including score pits⁴ (e.g. Section A boards).

For each candidate initial game state, we evaluate the game value using the MTD(f) algorithm [11,5]. “Game value” here has more information than win/lose/draw; it is the score difference at game end with optimal play. Search was highly optimized with the

⁴ When we have a fair game with piece(s) in the score pit, we have essentially created a fair game with fewer piece(s) and a perfect komi compensation for fairness.

MTD(f) algorithm, our large endgame database, and Irving’s node ordering: transposition table move, free moves right-to-left, captures right-to-left, and remaining moves right-to-left. For MTD(f), we used an initial game value guess of 0, with Plaat’s “next” choice being that of Van Horssen’s Algorithm 2, and using Van Horssen’s “SAFE” choice for storing the new best move in the transposition table.

After computing all 3 fair boards needing one piece movement and all 320 fair boards needing two piece movements (Section A Appendix), we computed and stored all non-terminal states along all possible optimal lines of play for these 323 fair initial positions.

3 Analysis of Fair Optimal Game Trees

A *fair optimal game tree* (FOGT) is a subtree of a game tree where the root game state of the subtree has game value 0 (i.e. results in a draw with perfect play), and each state node retains only successor-state children with game value 0. That is, a fair optimal play tree contains best plays of a drawn (sub)game.

There are pros and cons of analyzing data for optimal decisions from all game tree nodes, which we do later in the paper. On the positive side, one gains a complete picture of optimal play for any game states. On the negative side, the distribution of states includes mainly states that optimal players would not allow. That is, the distribution of states is skewed towards irrelevance in optimal game play.

In contrast, FOGTs provide the opportunity for focused insight on optimal play. We can ask questions of a state distribution where *both* players are playing optimally, gaining insight to what the highest level of Kalah play looks like. In the following subsections, we share raw and derived features of our collected game states as well as insights from the data.

3.1 Features

Of special interest are statistical and machine learning insights on the choice of optimal plays themselves. For this exploration, we define features that we conjecture may be relevant to optimal play decision-making and/or node game value prediction. These include both raw state variables and derived variables:

- depth** the number of plays made so far in the game (not included in model input)
- player** the current player p , either 0 or 1 for first/“s”/south player or second/“n”/north player, respectively (not included in model input)
- pit_6, . . . , pit_1, score, pit_60, . . . , pit_10, score_0** the number of pieces in each pit starting with the current player’s leftmost play pit and proceeding counter clock-wise around the board
- counters, counters_0** the sum of pieces in the current player’s and opponent’s play pits, respectively
- free_moves, free_moves_0** the number of current player and opponent play pits, respectively with pit index n containing n pieces
- is_free_move_0** whether or not pit n has n pieces

is_closest_free_move_n whether or not a pit n is the closest free move to the score pit
capture_move_n number of pieces captured by playing pit n
move_count number of non-empty pits on the current player's side of the board
move_count_o number of non-empty pits on the opponent's side of the board
relative_mobility $\text{move_count} - \text{move_count_o}$
is_closest_move_n whether or not a pit n move is the closest move to the score pit
score_gain_n the number of pieces scored after pit n is moved
is_largest_capture_move_n whether or not a pit n move captures the largest number of pieces among all capture moves
largest_capture the largest number of pieces that the current player could capture
score_diff the current player's scored counters minus their opponent's scored counters
counter_diff the current player's in-play counters minus their opponent's in-play counters
is_exposed_n whether or not pit n has no pieces across from it
piece_threat_n number of pieces in pit n under immediate threat of capture
game_val The current player score minus the opponent score if the game was played to completion with perfect play (0 for fair game states)
optimal_n whether or not pit n of the current player is an optimal move

3.2 Optimal Play Branching Factors and Tension

From the 63,050,664 unique non-terminal optimal play states of these MTD(f) searches (including optimal endgame database states), there are 195,748,684 total plays and 99,308,527 optimal plays, yielding average play and optimal play branching factors of approximately 3.1046 and 1.5751, respectively. The number of states with 1-6 optimal moves are, respectively: 36,156,747, 19,663,770, 5,446,419, 1,459,214, 298,957, and 25,557. Thus, 88.53% of states have 1 or 2 optimal plays. Most states with 6 optimal plays avoid playing into the opponent's play pits, with the opponent having a forced emptying of their pits on the next turn (e.g. 1 piece adjacent to their score pit).

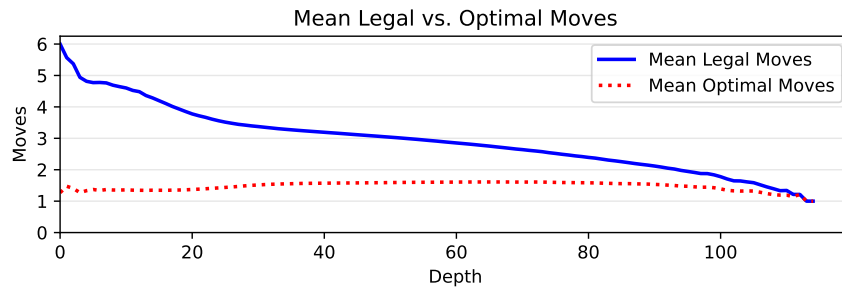


Fig. 2: Branching factor and optimal moves by depth.

For FOGT states, there is a decreasing trend in both branching factor and number of optimal moves with increasing depth, as shown in Figure 2. Whereas the branching

factor is always 6 at the root, the mean branching factor non-monotonically decreases to 1 as the number of pieces on the board decreases with depth. However, the mean number of optimal moves remains relatively steady, between 1 and 2 throughout play, slightly increasing for moderate depths.

As a result FOGT game tension, defined as the fraction of game-losing moves, is shown in Figure 3. This derives from Cameron Browne’s measure of puzzle tension [3], i.e. the fraction of losing plays. Here, we observe that tension is close to $\frac{5}{6}$ for root nodes, indicating that most but not all FairKalah roots have a unique optimal first play. This then non-monotonically decreases with depth down to 0.

A significant implication of this is that real-time management of play should favor computation distributed more towards early decisions. We would expect a precomputed opening book with a time-management policy such as OPEN [2] to be advisable for blunder avoidance when using limited memory and time, respectively.

3.3 Strategic Insights

Recall that a player gets a free move, i.e. another turn, when playing n pieces from pit n and having the last piece redistributed to the player’s score pit. Of the 63,050,664 unique non-terminal optimal play states, 35,810,907 (56.8%) offer free moves. Of these, 33,895,870 (94.65%) have the closest free move to one’s score pit as an optimal move. Thus, if one has one or more free moves, one should most likely play the closest free move to one’s score pit.

In the remaining 1,015,037 (5.35%) free-move states, the strategic considerations are diverse. In some states, taking the free move would force one to then play a closer pit that would spill one more pieces onto the opponent’s side. In others, it is preferable to accumulate pieces in low-index pits for endgame scoring. And, of course, some free moves spoil capture opportunities.

Captures are another important strategic consideration. Given that taking the closest free move is a frequent optimal play, we will now restrict our attention to states without free moves. There are 27,239,757 (43.2%) non-free-move states. Of these, 9,728,912 (35.27%) offer capture move(s). This is significantly lower than the game with empty captures where 85.21% of non-free-move states offer capture move(s)[9]. Without empty captures, of these non-free-move states with potential capture(s), 7,810,858 (80.28%) have an optimal capture move. Thus, if there is no free move, there is a 28.31% chance that a capture move is optimal.

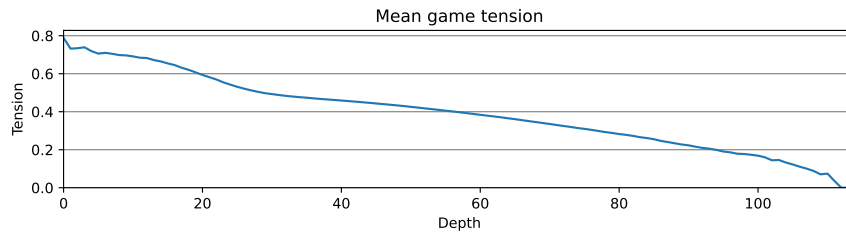


Fig. 3: Game tension by depth.

It is interesting to observe the distribution of pieces in last-turn states. We analyzed positions one move away from a terminal state. From Fig. 4 and Fig. 5, we can see that the players most often choose to hoard pieces in pit 1. Looking at the longest game possible [10], we can see the hoarding plays during the midgame, where player 1 eventually hoards 25 pieces in pit 1.

Pit	75th	90th	95th	99th	Max
Pit 1	1	4	5	8	18
Pit 2	1	2	3	5	17
Pit 3	1	2	3	5	16
Pit 4	1	2	3	5	14
Pit 5	1	2	3	5	13
Pit 6	1	2	2	4	10

Fig. 4: Stone Count Percentiles per Pit in Near-Terminal States

Pit	75th	90th	95th	99th	Max
Pit 1	2	4	5	10	19
Pit 2	1	3	4	6	18
Pit 3	1	2	4	6	15
Pit 4	1	3	4	6	14
Pit 5	1	2	3	6	13
Pit 6	1	2	3	4	11

Fig. 5: Stone Count Percentiles per Opponent Pit in Near-Terminal States

This brings us to the consideration of the frequencies with which play pits are found to be optimal:

Table 2: Number and percentage of states where each pit is optimal

Pit	States Optimal	Percent
1	24,617,195	39.04%
2	18,354,089	29.11%
3	15,956,613	25.30%
4	14,552,668	23.08%
5	14,035,154	22.26%
6	11,792,808	18.70%

As we can see, pits closer to a player’s score pit are indeed more likely to be optimal plays. The probability of optimality of the closest pit is highest and that probability decreases monotonically with each farther pit to slightly over half of the probability in the farthest pit. This is to be expected as close free moves are the most frequent and easy to set up, and one often clears the farthest pits in response to capture threats.

Note that our FOGT data fully supports the wisdom of the node-ordering heuristic of [6]: After a stored transposition table move (if available), consider free moves, then captures, then other moves, preferring moves closer to the score pit within each category. This should, in turn, guide our strategic prioritization of move considerations in play.

4 Modeling Optimal Play for Diverse States

We are interested in using such data to build predictors of expected game value as well as optimal move prediction by pit. While examining optimal game states of Kalah led to some interesting results, the predictive power of models trained on only FOGT data did not generalize well to non-fair states. Thus, to better model optimal play, we generated a different training data set that included unfair (non-0-value) Kalah states.

To efficiently generate diverse state data, we randomly generated distinct starting nodes by randomly scoring 5 pieces from different pits. We then analyzed optimal play for each node using a breadth-first traversal limited to 100 nodes per depth, similar to a beam search. This process was repeated for different randomized initial states until 14,941,556 states were generated. After removing depth and player-number features and adding the aforementioned engineered features from the current player’s perspective, the number of unique states was 10,925,291.

4.1 Model Performance Summary

For all predictions, we used a 50-50 train-test split. For prediction of the negamax state game value `game_val`, we trained a number of different models: scikit-learn’s `LinearRegression`, `DecisionTreeRegressor`, and `RandomForestRegressor`, as well as the CatBoost `CatBoostRegressor`. `DecisionTreeRegressor` had a `min_samples_split=500000`, `RandomForestRegressor` had hyperparameters `n_estimators=100`, and `CatBoostRegressor` had `iterations=30`, `learning_rate=0.9`, and `depth=16`. A performance summary is given in Figure 6.

Model	MSE	R ²
Linear regression	26.48	0.75
Decision tree	8.38	0.92
Random forest	3.83	0.96
CatBoost	2.79	0.97

Fig. 6: Regression model performance summary

Similarly, we built classification models for each play pit predicting `optimal_n`, i.e. whether the pit is an optimal play using scikit-learn’s `LogisticRegression` and `RandomForestClassifier`, as well as the `CatBoostClassifier`.

Since empty play pits are illegal for play and trivial to predict as not optimal, our performance measurements per pit are for only states where those pits are not empty. Accuracy and log loss performance results are given per pit number in Figures 7 and 8, respectively. We also include a “Base predictor” row for a probabilistic prediction based on the frequency of non-empty pit states for which that pit is optimal.

Model	6	5	4	3	2	1
Base predictor	.63	.73	.76	.79	.82	.85
Logistic regression	.80	.80	.79	.77	.78	.89
Random forest	.87	.87	.87	.87	.88	.92
CatBoost	.89	.90	.89	.89	.89	.94

Fig. 7: Classification accuracies per non-empty pit model

Model	6	5	4	3	2	1
Base predictor	.66	.59	.55	.51	.47	.43
Logistic regression	.43	.44	.47	.48	.46	.26
Random forest	.30	.29	.30	.29	.27	.17
CatBoost	.25	.25	.25	.25	.24	.15

Fig. 8: Classification log loss per non-empty pit model

4.2 SHAP Beeswarm Figure Interpretation

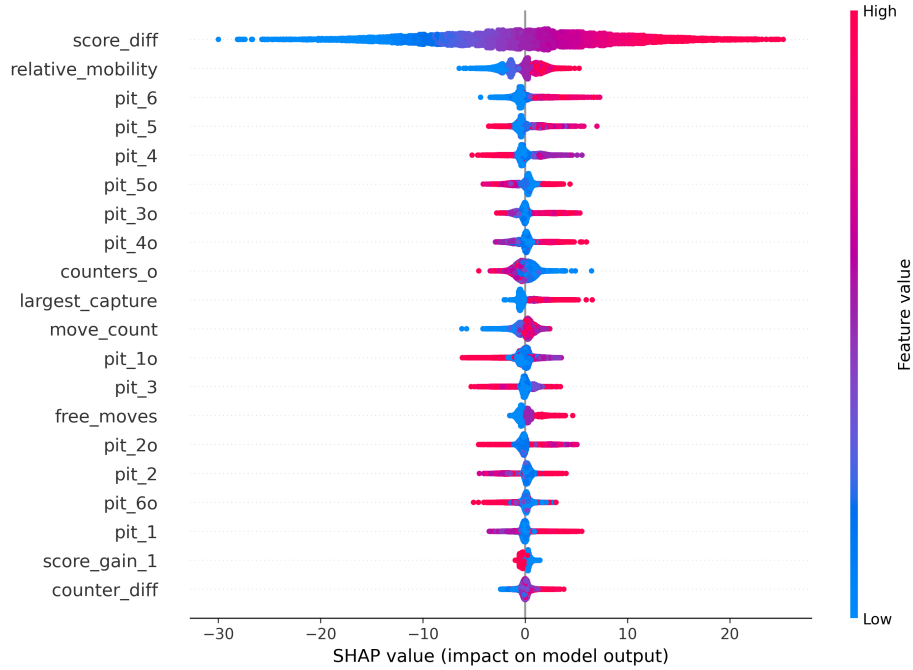


Fig. 9: SHAP beeswarm plot for CatBoost model predicting `game_val`

Using the `shap` package, we generated beeswarm plots in Figures 9 and 10, to visualize SHapley Additive exPlanation (SHAP) values [8] for our CatBoost `game_val` and `optimal_1` prediction models, respectively. SHAP values “assign each feature an importance value for a particular prediction.” [7] Each instance in our dataset represents one point for each horizontally visualized feature. The subset of features included are the most significant to the model prediction according to SHAP values, with higher features being most significant. The horizontal position of each point is determined by its SHAP value for that feature, and points are bunched more thickly along each feature

row to show density. Color of a dot represent the original feature value for that instance, with red/blue indicating high/low values, respectively.

For example, Figure 9 shows us first and foremost that `score_diff`, i.e., the current score difference, is the strongest positively correlating predictor for the final score difference, i.e. the game value. With limited capture in this rule set, having more `relative_mobility`, i.e., more legal moves to choose from, is very important. Having more mobility allows the player to have more positional potential, especially when one can not perform empty capture. Several individual pit features in both the current player and the opponents are important indicative for the game value. Pits like 6, 5, 4 (pits that are not close to the score pit), and pits $\bar{4}$, $\bar{5}$, $\bar{3}$ are strong predictors of the game value. We can see if the current player has more pieces in those pits, then they will gain advantage in those states, except for pit 4. Interestingly, if we look at pits 4, 3, $\bar{4}$, $\bar{3}$, we can see that it is a disadvantage to have more pieces in those pits for the respective player.

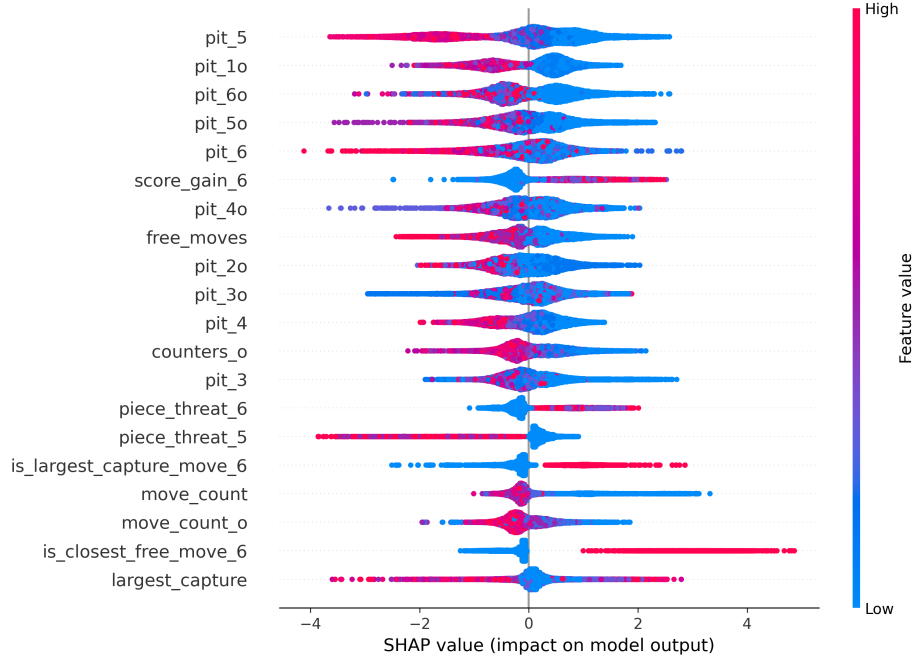


Fig. 10: SHAP beeswarm plot for CatBoost model predicting `optimal_6`

Turning our attention to Figure 10, we want to investigate the optimal move for pit 6. We chose to have this pit here in the paper since we saw that pit 6 is the least popular choice for optimal move, as we see in Table 2. We find that the single most significant positive predictor of pit 6 being optimal is that it is the closest free move. In contrast, the number of pieces in pit 6 is negatively associated with optimality: positions in which pit

6 holds many pieces tend to reduce the likelihood that the model identifies it as optimal. This aligns with trends observed across all pits: free-move proximity is the strongest positive driver, whereas higher stone counts consistently have a negative effect.

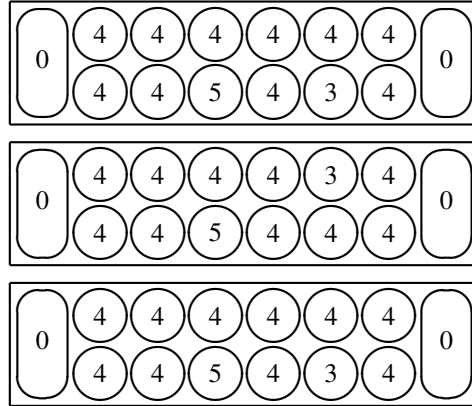
5 Conclusions

In this article, we have offered three primary contributions. First, we have analyzed fair starting states for Kalah (a.k.a. Mancala) *without empty captures* that should improve the game experience for future generations. Second, we have computed optimal play for different board configurations. Third, we have analyzed the fair optimal game trees to observe characteristics of perfect play. This again affirmed the node-ordering heuristic of [6], prioritizing free moves, then capture moves, then other moves, each categorization with a preference towards moves closer to the score pit. Finally, we collected optimal play data from a diverse set of optimal and suboptimal game states, yielding models with strength for predicting game values and optimal moves. These yielded further insights into important features for AI gameplay heuristics.

A Appendix: FairKalah Boards

A.1 Moving 1 Piece

These are all existing FairKalah boards where one moves exactly one piece from 4-pieces-per-pit initial conditions.



A.2 Moving 2 Pieces

All 251 existing FairKalah boards where one moves exactly two pieces from 4-pieces-per-pit initial conditions are available in SVG image form and as CSV data from the FairKalah website [10]. In the CSV data, comma-separated integers on each line indicate the number of pieces in each pit (including scoring pits) starting from the first (i.e. “south”) player’s leftmost play pit and proceeding counterclockwise.

References

1. Games: Pits & Pebbles. Time p. 67 (Jun 1963), <https://time.com/archive/6812568/games-pits-pebbles/>
2. Baier, H., Winands, M.H.M.: Time management for Monte-Carlo tree search in Go. In: van den Herik, H.J., Plaat, A. (eds.) *Advances in Computer Games*. pp. 39–51. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
3. Browne, C.: Tension in puzzles. *Game & Puzzle Design Journal* **3**(2), 71–78 (2017)
4. Champion, W.J.: Game counter (US Patent US2720362A, 1955-10-11), <https://patents.google.com/patent/US2720362A/en>
5. van Horssen, J.J.: Move selection in MTD(f). *ICGA Journal* **41**, 1–9 (03 2019). <https://doi.org/10.3233/ICG-190096>
6. Irving, G., Donkers, J., Uiterwijk, J.: Solving Kalah. *ICGA Journal* **23**(3), 139–147 (September 2000), <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.4.7870>
7. Lundberg, S.: beeswarm plot – SHAP latest documentation. https://shap.readthedocs.io/en/latest/example_notebooks/api_examples/plots/beeswarm.html (2022), accessed: 2022-09-03
8. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (eds.) *Advances in Neural Information Processing Systems*. vol. 30. Curran Associates, Inc. (2017), <https://proceedings.neurips.cc/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf>
9. Neller, T.W., Neller, T.C.: Fairkalah: Towards fair Mancala play. In: *Computers and Games: International Conference, CG 2022, Virtual Event, November 22–24, 2022, Revised Selected Papers*. pp. 3–13. Springer-Verlag, Berlin, Heidelberg (2022). https://doi.org/10.1007/978-3-031-34017-8_1
10. Neller, T.W., Pham, P.: FairKalah: Fair Mancala. <https://www.cs.gettysburg.edu/~tneller/games/fairkalah/>, accessed: 2025-12-07
11. Plaat, A., Schaeffer, J., Pijls, W., de Bruin, A.: Best-first fixed-depth minimax algorithms. *Artificial Intelligence* **87**(1), 255–293 (1996). [https://doi.org/https://doi.org/10.1016/0004-3702\(95\)00126-3](https://doi.org/https://doi.org/10.1016/0004-3702(95)00126-3)
12. Russ, L.: *The Complete Mancala Games Book: How To Play the World’s Oldest Board Games*. Marlowe & Company (2000), <https://books.google.com/books?id=BA9yHQAACAAJ>
13. Silver, R.: The “Kalah” game playing program. Digital Equipment Corporation interoffice memorandum M-1098 via PDP Distribution List (March 1961), http://bitsavers.informatik.uni-stuttgart.de/pdf/dec/pdp1/memos/M-1098_Kalah_Mar61.pdf